HDOC-54

Maximum Sum of Contiguous Subarrays of Size k Using Pointers

Question

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k < size. Pass the array to a function named max_subarray_sum() using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function must identify all contiguous subarrays of size k in arr[], calculate the sum of the elements in each subarray, find and display the maximum subarray sum, and display -1 if k is invalid.

Algorithm

1. Start the program.
2. Read the size of the array and then read all array elements.
3. Read the value of k.
4. Call max_subarray_sum(arr, size, k), where arr is passed as the address of the first element.
5. Inside the function, check whether k is valid. If k <= 0 or k >= size, display -1 and return.
6. For every possible starting index i from 0 to size - k, calculate the sum of k consecutive elements using pointer arithmetic.
7. Display each contiguous subarray and its sum.
8. Keep track of the largest sum found.
9. Display the maximum subarray sum.
10. Stop the program.

C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int size, int k)
{
   int i, j, sum, maxSum;

   if (k <= 0 || k >= size)
   {
      printf("-1\n");
      return;
   }

   maxSum = 0;
   for (j = 0; j < k; j++)
      maxSum += *(arr + j);

   printf("\nContiguous subarrays of size %d and their sums:\n", k);

   for (i = 0; i <= size - k; i++)
   {
      sum = 0;
      printf("[");

      for (j = 0; j < k; j++)
      {
         printf("%d", *(arr + i + j));
         if (j < k - 1)
            printf(" ");
```

```
            sum += *(arr + i + j);
        }

        printf("] -> Sum = %d\n", sum);

        if (sum > maxSum)
            maxSum = sum;
    }

    printf("Maximum subarray sum = %d\n", maxSum);
}

int main()
{
    int size, k, i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Enter k: ");
    scanf("%d", &k);

    max_subarray_sum(arr, size, k);

    return 0;
}
```

Explanation

The array name arr represents the address of its first element, so it is passed to max_subarray_sum() as an int pointer. Inside the function, *(arr + i + j) accesses array elements using pointer arithmetic.

For every valid starting position, the program examines exactly k consecutive elements. Their sum is calculated and displayed. maxSum stores the greatest sum encountered.

The condition 0 < k < size is checked before processing. If k is outside this range, the function displays -1 as required.

Test Case Table

Test	Size	Array Elements	k	Subarray Sums / Condition	Output
1	5	1 2 3 4 5	2	[1 2]=3, [2 3]=5, [3 4]=7, [4 5]=9	9
2	6	-2 5 -1 7 -3 4	3	[-2 5 -1]=2, [5 -1 7]=11, [-1 7 -3]=3, [7 -3 4]=8	11
3	4	10 -2 3 1	1	[10]=10, [-2]=-2, [3]=3, [1]=1	10
4	5	2 4 6 8 10	5	Invalid k because k >= size	-1
5	5	2 4 6 8 10	0	Invalid k because k <= 0	-1

Sample Output

```
Enter the size of the array: 6
Enter 6 array elements: -2 5 -1 7 -3 4
Enter k: 3

Contiguous subarrays of size 3 and their sums:
[-2 5 -1] -> Sum = 2
[5 -1 7] -> Sum = 11
[-1 7 -3] -> Sum = 3
[7 -3 4] -> Sum = 8
Maximum subarray sum = 11
```

Result

The C program was implemented successfully using pointers to pass and access the variable-length integer array. It displays all contiguous subarrays of size k with their sums, reports the maximum subarray sum, and displays -1 when k is invalid.